

(either distributed or standalone). Source connectors push data into Kafka, while sink connectors create a bridge-like configuration that sends data out to other systems. With this configuration, creating end-to-end data pipelines is easy and requires minimal custom code.

How Kafka Connect fits into a modern pipeline: Kafka Connect is a crucial component of contemporary data pipelines since it provides a simple and trustworthy bridge between Kafka and other systems, including databases, cloud storage, or analytics software. Kafka Connect handles data into and out of Kafka via source connectors to pull data in and sink connectors to push data out. In addition, it is compatible with real-time data processing engines such as Apache Flink, Spark, and ksqlDB, and accommodates both ETL and ELT setups. Kafka Connect's flexibility, fault tolerance, and simplicity make it substantially easier to build high-performing, open source data pipelines with minimal custom code.

Best practices for using Kafka connectors

Deployment and configuration: For production workloads, Kafka Connect must be run in distributed mode to provide redundancy and horizontal scalability. Keep all connector configurations in version-controlled files for consistency and auditing purposes. When processing structured data formats such as Avro or JSON, integrate with a Schema Registry to enforce schemas and avoid data corruption. Optimise performance parameters such as `batch.size`, `poll.interval.ms`, and `max.task` for optimal throughput. Implement Dead Letter Queues (DLQs) to catch and examine records that do not process. Deploy automatically using Helm, Docker, or Kubernetes to make operations easier.

Selecting the right connector: Selecting the appropriate Kafka connector depends on the source or sink system, data format, and scalability requirement. Use open source, well-supported connectors such as Debezium for CDC, JDBC for relational databases, and FileStream for file-based integration.

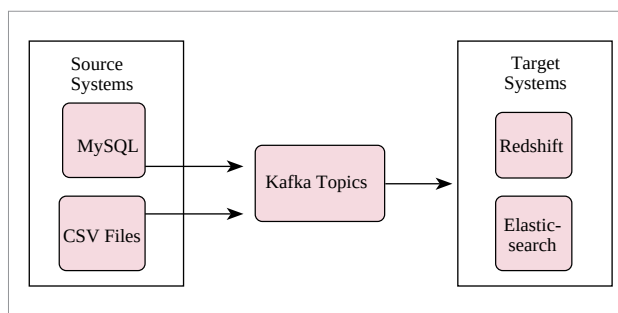


Figure 2: Kafka Connect in action

Analyse throughput, schema format compatibility, and fault-tolerance. Assess community support, update frequency, and documentation quality. Custom connectors may be required for complex or domain-specific requirements. Before being used in production, connectors should always be tested in a staging environment.

Observability, error handling, and scaling

Monitoring pipelines: Monitoring ensures data pipelines remain healthy and performant. Tools like Prometheus and Grafana can be used to track critical metrics such as:

- Connector health
- Task performance
- Throughput rates
- Failure counts

By setting up DLQs, problematic records can be safely isolated and inspected without stopping the pipeline. Additionally, Kafka Connect provides a REST API for examining logs and connector status.

Handling failures and rebalancing: Fault tolerance is built into Kafka Connect. If a connector task fails, it can automatically retry or restart based on the error policy. DLQs help ensure faulty records don't pollute downstream systems. The distributed nature of Kafka Connect allows automatic task redistribution across available workers during scaling events.

Table 1: Connector selection matrix

Connector	Use case	Throughput	Compatibility	Schema support	Fault tolerance	Community support
Debezium	Change data capture (CDC)	High	PostgreSQL, MySQL, MongoDB, etc	Avro, JSON, Protobuf	Strong	Very active
JDBC	Relational DB ingestion	Medium	Any JDBC-compliant DB	Avro, JSON	Moderate	Active
FileStream	Local file system integration	Low	Plaintext, JSON, CSV	Minimal	Basic	Limited
MQTT	IoT sensor stream ingestion	Medium	MQTT brokers	JSON	Moderate	Niche